

How tour guide works

Deep analysis generated 2026-09-19 · app booted via docker; screenshots are real captures · model claude-sonnet-5

Overview

Tour Guide is a personal, self-hosted trip-planning and audio-tour-guide web app for turning the dead time between an airport (or any starting point) and a hotel (or any ending point) into a curated half/full day of sightseeing. A traveler enters a start, an end, a date, an arrival time and a "be there by" deadline, and a free-text description of interests ("indian food, historic neighborhoods, upscale areas, street art"); the app asks Claude to propose a corridor of candidate stops, grounds every candidate against real Wikipedia articles and Nominatim/OSM geocoding, routes the day with Valhalla (falling back to OSRM), and produces a scheduled, lunch-aware, time-boxed itinerary that the user can edit, reorder, add to, or trim by hand. The result can be shared as a compressed link, opened in Google Maps, or handed off to "drive mode" — an installable, offline-capable PWA screen that follows the car by GPS, narrates each stop and points of interest passed along the way through the phone's own text-to-speech, and needs no server or network once prepared.

The product is aimed at one person planning their own road-trip day (the README calls it "personal project. First trip: Houston, November 2026"), but architecturally it separates cleanly into a planning phase that needs a laptop, an API key/Claude CLI, and internet access to Wikipedia/Nominatim/routing services, and a driving phase that needs only the pre-baked "drive package" stored in IndexedDB and live map tiles. This split is deliberate: prepare the trip ahead of time on a laptop, then install the PWA on a phone and import the exported JSON trip file so the actual drive works with only cellular data for tiles.

End to end, a session looks like: fill in start/end/date/times/interests on the plan screen → submit "Plan my tour" → watch a live streamed progress panel as Claude's candidate stops appear and are verified one by one against Wikipedia/Nominatim → get a routed, scheduled list of numbered stops on a map, with a "Didn't fit" list of trimmed/not-found candidates → tweak stops (reorder, remove, add via search or a map tap, "Suggest more", toggle lunch, adjust dwell) with every edit silently re-routing and re-scheduling but never auto-trimming → optionally click "Prepare drive (narration)" to have the server sample the route, geosearch Wikipedia for interesting drive-by places, and have Claude write short spoken scripts for every stop and drive-by → export the resulting "drive package" as JSON (or keep it on the same device) → open drive.html, import the trip if needed, tap "Start drive", and let the app follow GPS, fire geofenced narration, show a next-turn banner, and mark stops visited automatically.

The whole thing is designed around being a polite, low-budget consumer of public APIs (Wikipedia, Nominatim, Photon, Valhalla/OSRM, OSM tiles), rate-limiting and caching every one of them, and around working with either a paid Anthropic API key or a local Claude Code CLI subscription so a single person can run this cheaply for one real trip.

Architecture

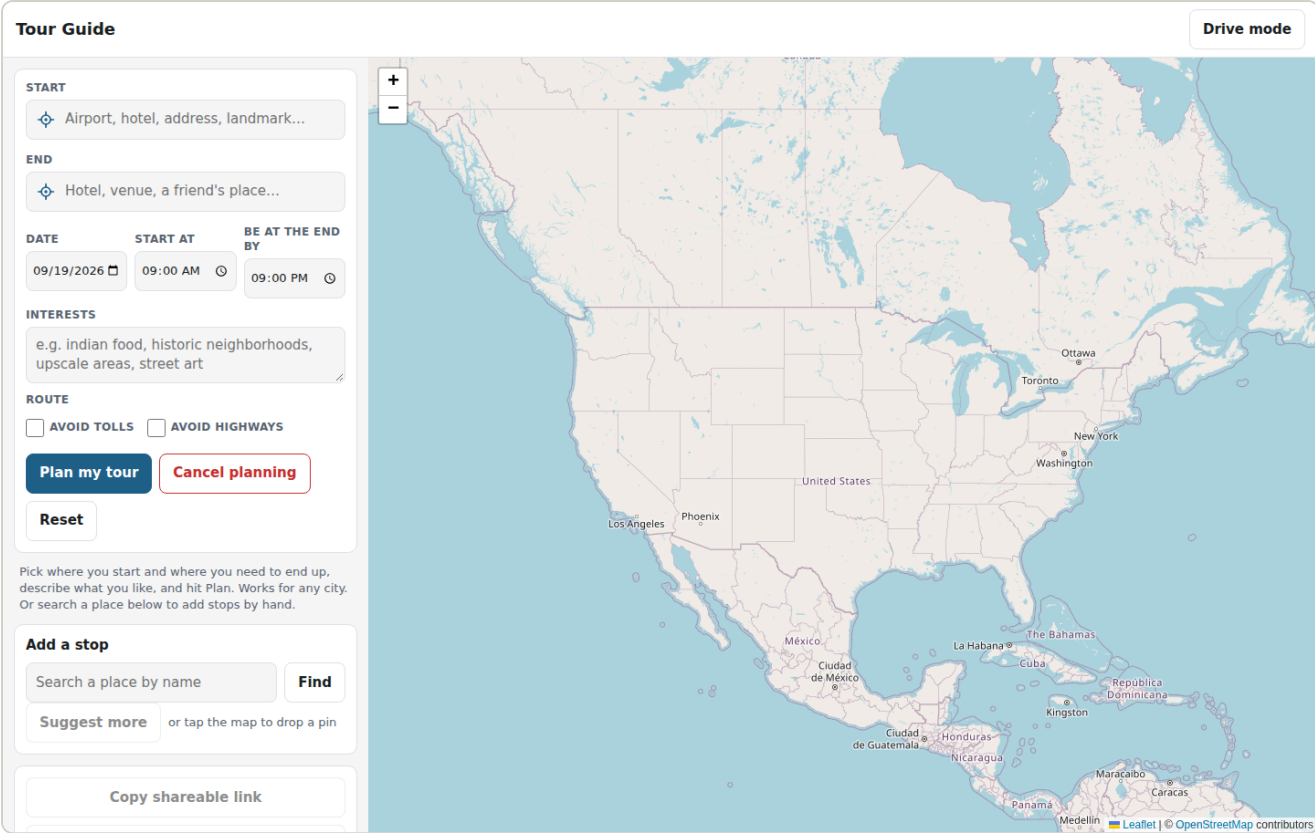
The backend is a small Express app (`server/index.js`) exposing a JSON/SSE API under `/api/*`: `GET /api/health`, `GET /api/place`, `GET /api/reverse`, `POST /api/plan` (streamable), `GET /api/estimate`, `POST /api/schedule`, `POST /api/suggest`, and `POST /api/prepare-drive` (streamable). Planning (`server/plan.js`) orchestrates three stages — Claude proposes candidates (`server/claude.js`, using the Anthropic SDK if `ANTHROPIC_API_KEY` is set, otherwise shelling out to the `claude` CLI), the server grounds each candidate against Wikipedia and Nominatim (`server/resolve.js`, `server/wikipedia.js`, `server/nominatim.js`), and routing/scheduling runs through Valhalla or OSRM (`server/router.js`, `server/schedule.js`) using pure math shared with the browser (`web/js/schedule-core.js`, `web/js/routeMath.js`, also vendored server-side via relative import). A trimming/compression algorithm (in `schedule-core.js`, driven from `server/schedule.js`) removes or shortens stops to fit the deadline. `server/narrate.js` builds the drive package: it reuses the planned route, samples it every 6 km to geosearch Wikipedia for nearby articles (`server/lib/wikiGeo.js`), filters/scores/dedupes candidates, and asks Claude for narration text, streaming progress the whole way. Small libs (`server/lib/http.js`, `server/lib/queue.js`, `server/lib/cache.js`, `server/lib/timings.js`,

`server/lib/geo.js`) provide fetch-with-retry, serial rate-limited queues (Nominatim/OSRM at 1 req/s), a TTL cache (24 h Wikipedia, 7-day Nominatim), and a learned-median time-estimator persisted to `data/timings.json`.

The frontend (`web/`) is a static, no-build PWA. `web/js/state.js` holds a single in-memory `Itinerary` object with a pub/sub `subscribe`; `web/js/actions.js` mutates it (add/remove/move/update stop, toggle lunch, set route options) and debounced-reschedules through `POST /api/schedule` whenever something changes, while `web/js/itinerary.js` renders the sidebar/stop cards and `web/js/map.js` renders the Leaflet map, markers and route polyline. `web/js/api.js` wraps `fetch`, including a generic SSE parser used for both streaming endpoints, and handles the optional `x-app-key` passphrase flow for hosted deployments (`APP\_SECRET`). Planning progress and prepare-drive progress are driven by `web/js/actions.js` / `web/js/drivePrep.js` consuming those SSE events and updating a shared "busy" UI (`web/js/busy.js`) with a learned ETA (`web/js/timings.js`, mirrored server-side). Drive mode (`web/drive.html` + `web/js/drive.js`) is an almost fully separate, offline-first app: it loads a previously prepared "drive package" from IndexedDB (`web/js/storage.js`), projects GPS fixes onto the route (`web/js/routeMath.js`), and drives a pure, unit-tested geofence state machine (`web/js/geofence.js`) and a speech queue (`web/js/speech.js`) built on the Web Speech API, with an optional GPS simulator (`web/js/sim.js`) for testing without driving. A service worker (`web/sw.js`) caches the app shell and vendored Leaflet for install/offline use, caches OSM tiles only once requested (per OSM tile policy), and never caches `/api/\*`.

Data flows in one direction for planning (form → `state.js` → `POST /api/plan` (SSE) → grounded/routed/scheduled `Itinerary` → back into `state.js` → re-rendered by `itinerary.js`/`map.js`), and edits flow the same loop through `POST /api/schedule`. Drive mode flow is local-only after preparation: `Itinerary` + `narration[]` (the `DrivePackage`) is written once via `POST /api/prepare-drive`, persisted to IndexedDB, and thereafter read-only, driven purely by GPS fixes and the pure `geofence`/`routeMath` modules with no server calls.

The main planning and editing screen: collect a start, end, schedule and interests; run Claude-driven planning; and let the traveler adjust, share, or hand off the resulting day plan.



Walkthrough

A first-time visitor sees a map (top or left, depending on viewport) and a sidebar form with Start and End fields, a date and two time pickers (arrival "Start at" and "Be at the end by"), an Interests textarea, and Avoid tolls/Avoid highways checkboxes. Typing into Start or End triggers a Photon-backed type-ahead (also matching bundled airport codes like "SNA"); tapping the location icon instead fills in the browser's current GPS position and reverse-geocodes it to a friendly label. Submitting "Plan my tour" replaces the form's message area with a live-updating list of candidate stops as Claude proposes them and the server verifies each one against Wikipedia/Nominatim, then finally renders a numbered, timed list of stops on the map and in the sidebar, with a running trip summary and a "Didn't fit / not found" section for anything cut or ungroundable. From there the user can search and add specific places, ask for "Suggest more" AI candidates, drag/tap the map to drop a pin, toggle which stop is lunch, and everything re-routes and re-times automatically without re-running Claude or re-trimming. Once satisfied, the user can copy a compressed shareable link, open the day as one or more Google Maps navigation links, or click "Prepare drive (narration)" to build the offline drive package, which can then be exported as a JSON file for import into the installed PWA on a phone.

Controls

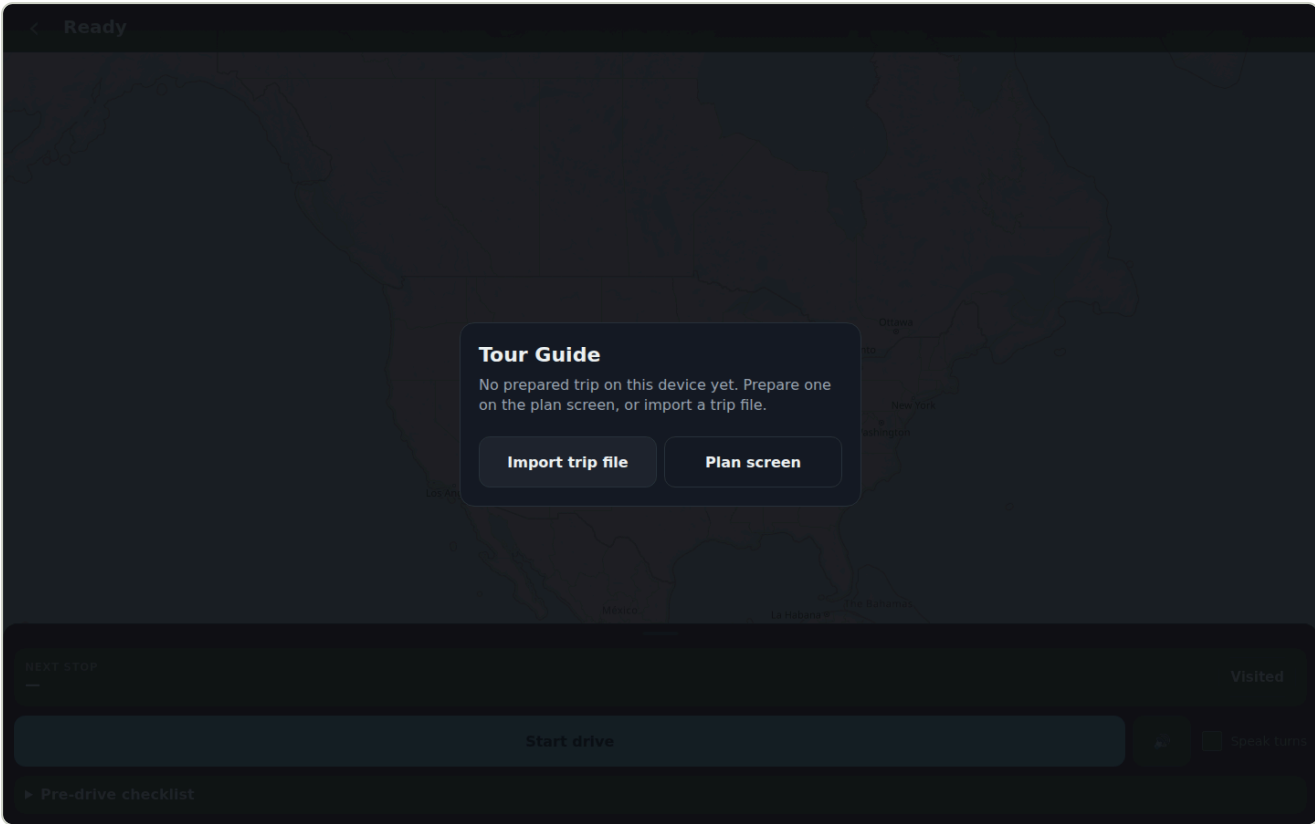
CONTROL	WHAT IT DOES
Drive mode	`<a href="drive.html">` navigates to `web/drive.html`, the driving screen; no state is passed except whatever trip is later imported/loaded from IndexedDB.

Zoom in / Zoom out	Default Leaflet zoom-control links created by <code>L.map(el, { zoomControl: true })` in <code>web/js/map.js`'s <code>init()`; they zoom the plan map in/out.</code></code></code>
Leaflet / OpenStreetMap	Leaflet's built-in attribution control links (the "Leaflet" credit and the <code>&copy; OpenStreetMap contributors` attribution string attached to the <code>L.tileLayer` in <code>map.js`); purely informational, open the respective project sites.</code></code></code>
Use my current location as the start	<code>#start-loc` button bound in <code>web/js/itinerary.js` <code>bindEndpoint("start")`; calls <code>navigator.geolocation.getCurrentPosition`, sets <code>label:"Current location" via <code>actions.setStart`, then refines the label via <code>typeahead.reverseLabel` (Photon reverse geocode) and re-applies it.</code></code></code></code></code></code></code>
Airport, hotel, address, landmark ... (start input)	<code>#start-query`, wired to <code>typeahead.attach()` in <code>itinerary.js`; queries Komoot Photon plus the bundled <code>web/data/airports.json` as the user types, and on pick calls <code>actions.setStart({label, lat, lon})`, which patches state and calls <code>actions.reschedule()`.</code></code></code></code></code></code>
Use my current location as the end	<code>#end-loc`, same handler as the start button but targeting <code>actions.setEnd` (<code>bindEndpoint("end")`).</code></code></code>
Hotel, venue, a friend's place... (end input)	<code>#end-query`, same type-ahead mechanism as the start field, calling <code>actions.setEnd` on pick.</code></code>
Date field (#date)	On <code>change`, <code>itinerary.js` calls <code>actions.setField({date: ...})` then <code>actions.reschedule()`, which debounced-calls <code>POST /api/schedule` to re-time the day for the new date (affects nothing but is carried through the itinerary).</code></code></code></code></code>
Start at / Be at the end by (time inputs #arrival, #deadline)	Each <code>change` calls <code>actions.setField` with <code>arrivalTime`/`deadline` then <code>actions.reschedule()`, re-running <code>server/schedule.js`'s <code>computeItinerary` (via <code>POST /api/schedule`) to re-walk arrival/departure times and lunch placement.</code></code></code></code></code></code></code>
Interests textarea (#interests)	Updates <code>state.interests` via <code>actions.setField` on change; deliberately does **not** reschedule — it only takes effect the next time "Plan my tour" or "Suggest more" is run.</code></code>
Avoid tolls / Avoid highways checkboxes	<code>#avoid-tolls`/`#avoid-highways` call <code>actions.setRouteOptions({avoidTolls avoidHighways})`, patching <code>routeOptions` and calling <code>actions.reschedule()`, which passes those flags to <code>POST /api/schedule` → <code>server/router.js` (Valhalla honors avoid-tolls/avoid-highways; falls back to OSRM, which cannot).</code></code></code></code></code></code>

Plan my tour (submit)	<code>`#plan-form`</code> submit handler in <code>`itinerary.js`</code> calls <code>`actions.plan()`</code> , which opens an <code>`AbortController`</code> , calls <code>`api.planStream()`</code> → <code>`POST /api/plan`</code> with <code>`Accept: text/event-stream`</code> , driving <code>`server/plan.js`</code> 's <code>`runPlan()`</code> (Claude propose → Wikipedia/Nominatim ground → route → schedule/trim), streaming <code>`estimate`/`phase`/`candidates`/`stop`/`dropped`/`done`/`error`</code> events into the sidebar progress display before replacing <code>`state`</code> with the final itinerary.
Cancel planning	<code>`#cancel-plan-btn`</code> calls <code>`actions.cancelPlan()`</code> , which aborts the <code>`AbortController`</code> used by <code>`plan()`</code> ; the server's <code>`res.on("close", ...)`</code> handler in <code>`server/index.js`</code> notices the dropped connection, aborts the Claude call, and stops work server-side too.
Reset	<code>`#reset-btn`</code> , after a <code>`confirm()`</code> dialog, calls <code>`actions.reset()`</code> to clear the itinerary back to <code>`state.emptyItinerary()`</code> and clears the start/end query inputs.
Search a place by name (#add-query)	Free-text input for the "Add a stop" search block; combined with the "Find" button via <code>`bindSearch()`</code> in <code>`itinerary.js`</code> .
Find	<code>`#add-search`</code> triggers <code>`bindSearch`</code> 's <code>`go()`</code> , which calls <code>`searchPlace()`</code> → <code>`api.place(q, near)`</code> → <code>`GET /api/place?q=&near=lat,lon`</code> , handled by <code>`server/index.js`</code> 's <code>`/api/place`</code> route (<code>`stops.searchPlace`</code> , explicit-submit only, never autocomplete per Nominatim policy), and renders results with an "Add" action that calls <code>`actions.addStop()`</code> .
Suggest more	<code>`#suggest-btn`</code> requests a few more grounded candidates not already on the itinerary via <code>`api.suggest(itinerary, count)`</code> → <code>`POST /api/suggest`</code> , handled by <code>`server/index.js`</code> , which calls <code>`claude.suggestMore()`</code> then <code>`resolve.resolveCandidates()`</code> before filtering out anything already present; results render into <code>`#suggest-results`</code> for the user to add.
Copy shareable link	<code>`#share-btn`</code> , bound in <code>`web/js/share.js`</code> <code>`bind()`</code> ; compresses a trimmed copy of the itinerary (<code>`compact()`</code> , dropping route geometry, truncating long text) with <code>`CompressionStream("deflate-raw")`</code> , base64url-encodes it into a <code>`#i=...`</code> URL hash, and copies the full URL to the clipboard.
Open in Google Maps	<code>`#gmaps-btn`</code> , also in <code>`share.js`</code> ; splits the trip into Google Maps directions URLs respecting Google's waypoint caps (<code>`googleMapsLegs`</code>), opening a single tab directly if one leg suffices or listing per-leg links in <code>`#gmaps-legs`</code> otherwise.
Prepare drive (narration)	<code>`#prepare-btn`</code> (bound in <code>`web/js/drivePrep.js`</code> <code>`bind()`</code>) calls <code>`prepare()`</code> , which opens an <code>`AbortController`</code> , streams <code>`POST /api/prepare-drive`</code> via <code>`api.prepareDriveStream()`</code> , drives <code>`server/narrate.js`</code> 's <code>`prepareDrive()`</code> (route reuse, Wikipedia corridor scan via <code>`server/lib/wikiGeo.js`</code> , one Claude call for narration scripts), shows live phase/scan/candidate/narration progress, and on success saves the resulting <code>`DrivePackage`</code> to IndexedDB via <code>`storage.saveTrip()`</code> .
Cancel (prepare)	<code>`#cancel-prepare-btn`</code> aborts the in-flight <code>`prepare()`</code> controller, causing <code>`drivePrep.js`</code> to throw a <code>`cancelled`</code> error that the UI turns into a "Prepare cancelled." toast; the server's connection-close handler aborts its Claude call too.
Export trip	<code>`#export-btn`</code> calls <code>`drivePrep.exportPackage(pkg)`</code> , which serializes the current <code>`DrivePackage`</code> to a JSON <code>`Blob`/`File`</code> named <code>`trip-<date>.json`</code> and either invokes the Web Share API (<code>`navigator.share`</code>) or triggers a browser download link.

Page: /drive.html

The offline, GPS-driven "drive mode" screen that follows the car along the prepared route and narrates stops and points of interest aloud, without needing the planning server.



Walkthrough

Arriving at this page, `web/js/drive.js`'s `boot()` initializes a dark-themed Leaflet map and tries to load a trip: from a `?trip=` query param, the last-used trip in IndexedDB, or — if there are multiple or none — an overlay listing saved trips plus an "Import trip file" button and a "Plan screen" link back to `index.html`. Once a package is selected, the route, numbered stop markers, and drive-by point markers are drawn, and the bottom sheet shows the next stop card, drive controls, and a collapsible pre-drive checklist (location permissions, Auto-Lock, ringer, phone mount, CarPlay). Tapping "Start drive" synchronously speaks "Starting tour" (to satisfy iOS's user-gesture requirement for audio), acquires a screen Wake Lock, and starts `watchPosition`; from then on the car icon tracks GPS, the top banner shows either the current road/turn or an off-route arrow toward the next stop, and narration fires automatically via the geofence engine — a stop interrupts a queued drive-by, drive-bys wait, and each piece of text is read sentence-by-sentence with replay/skip controls in the "Now playing" card. Stops are marked visited automatically after dwelling 20 seconds inside their radius, on leaving the radius, when route progress passes them by 1.2 km, or by tapping "Visited" manually; all fired/visited state is persisted to IndexedDB so a reload mid-drive does not replay narration. If launched with `?sim=1`, a hidden simulation panel (not part of this crawl's visible controls) replaces GPS with a scripted replay for testing.

Controls

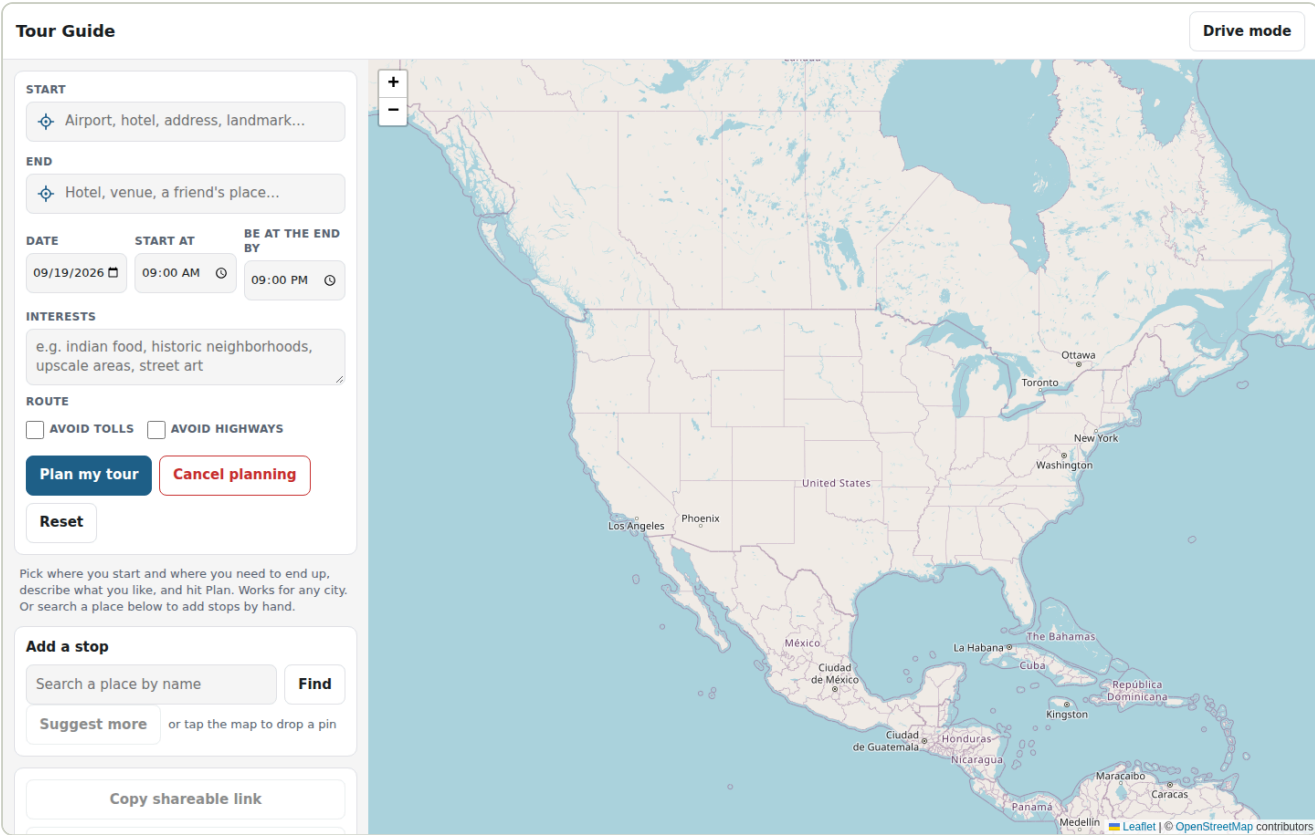
CONTROL	WHAT IT DOES
Leaflet / OpenStreet	Leaflet's built-in attribution links from the <code>L.tileLayer</code> added in <code>drive.js</code> 's <code>initMap()</code> ; open the respective project sites, same mechanism as the plan screen's map.

tMap

Back to plan	<code></code> in the banner header; a plain link back to the planning screen (<code>web/index.html</code>), no state transfer beyond whatever trip already sits in IndexedDB.
Visited	<code>#skip-stop</code> button ("Mark visited") in the "Next stop" card; manually marks the currently-tracked next stop as visited (via the geofence module's <code>markVisited</code>), advancing <code>state.nextStopIdx</code> and updating the stop marker styling, for cases where the automatic dwell/leave/progress detection in <code>web/js/geofence.js</code> doesn't trigger.
Start drive	<code>#start-btn</code> ; per the README and <code>drive.js</code> , performs three actions synchronously inside the tap handler — <code>speech.unlock("Starting tour")</code> to prime iOS text-to-speech, <code>navigator.wakeLock.request()</code> to keep the screen on, and <code>navigator.geolocation.watchPosition()</code> to begin tracking — then drives the on-road loop (route projection via <code>routeMath.project</code> , geofence updates, banner/turn display) for the remainder of the drive.
 (mute toggle)	<code>#mute-btn</code> , <code>aria-pressed</code> toggle; calls the speech module's <code>setMuted()</code> to silence/unsilence narration while still advancing the geofence/queue timing logic (so nothing gets skipped, it just plays silently at reading pace).
Speak turns (checkbox, unlabeled in crawl = #speak-turns)	Toggles whether the next-turn banner (derived from OSRM/Valhalla route steps) is also read aloud; off by default because, per the README, the car's own navigation app usually announces turns better.
Pre-drive checklist (summary)	<code><details><summary></code> disclosure widget; expands/collapses a plain informational checklist (location permission, Auto-Lock, ringer, phone mount/charging, CarPlay/Android Auto, tap Start and listen for the chime) with no side effects beyond showing/hiding the <code></code> .
Plan screen	<code></code> inside the <code>#overlay</code> trip picker shown when no trip is loaded; links back to the planning screen so the user can plan or re-prepare a trip.

Page: /index.html

Identical document to `/` — the same planning and editing screen, served at its explicit filename.



Walkthrough

Behaves exactly as described for `/`: same markup, same `web/js/main.js` boot sequence (`map.init`, `itinerary.bindForm`, `share.bind`, `drivePrep.bind`, hash-based trip restore, health probe against `GET /api/health`, service worker registration), and the same sidebar/map/plan/edit/share/prepare-drive flow.

Controls

CONTROL	WHAT IT DOES
Drive mode	Same as `/` — ` <a href="drive.html">></a> ` to the drive screen.
Zoom in / Zoom out	Same Leaflet zoom control as `/`.
Leaflet / OpenStreetMap	Same Leaflet/OSM attribution links as `/`.
Use my current location as the start	Same `#start-loc` handler as `/` (`itinerary.js` `bindEndpoint("start")` → `actions.setStart`).
Airport, hotel, address, landmark...	Same `#start-query` type-ahead as `/`.
Use my current location as the end	Same `#end-loc` handler as `/` (`actions.setEnd`).

Hotel, venue, a friend's place...	Same <code>`#end-query`</code> type-ahead as <code>`/`</code> .
Date / time fields	Same <code>`change`</code> → <code>`actions.setField`</code> + <code>`actions.reschedule()`</code> behavior as <code>`/`</code> .
Interests textarea	Same <code>`actions.setField`</code> (no reschedule) behavior as <code>`/`</code> .
Route checkboxes (Avoid tolls / Avoid highways)	Same <code>`actions.setRouteOptions()`</code> + <code>`actions.reschedule()`</code> behavior as <code>`/`</code> .
Plan my tour	Same <code>`actions.plan()`</code> → <code>`POST /api/plan`</code> (SSE) flow as <code>`/`</code> .
Cancel planning	Same <code>`actions.cancelPlan()`</code> behavior as <code>`/`</code> .
Reset	Same <code>`actions.reset()`</code> behavior as <code>`/`</code> .
Search a place by name / Find	Same <code>`GET /api/place`</code> search-and-add flow as <code>`/`</code> .
Suggest more	Same <code>`POST /api/suggest`</code> flow as <code>`/`</code> .
Copy shareable link	Same <code>`share.js`</code> compressed-hash-link flow as <code>`/`</code> .
Open in Google Maps	Same <code>`share.js`</code> Google Maps leg-splitting flow as <code>`/`</code> .
Prepare drive (narration)	Same <code>`drivePrep.prepare()`</code> → <code>`POST /api/prepare-drive`</code> (SSE) flow as <code>`/`</code> .
Cancel (prepare)	Same abort-in-flight-prepare behavior as <code>`/`</code> .
Export trip	Same <code>`drivePrep.exportPackage()`</code> download/share flow as <code>`/`</code> .

Generated by the workbench deep-analysis pipeline from the repository source and a live crawl.